

多模态生成技术优化实践第一期—— 稀疏与量化篇

作者：陈泓仰、赵玺

时间：2026.5.13

目录

Part 1 背景

Part 2 块稀疏注意力

Part 3 低比特量化FA

背景

业界背景



视频生成模型已从学术研究进入大规模商业落地阶段，短视频创作、广告制作、影视辅助生产等场景需求爆发。



产业竞争加速，闭源与开源模型同步演进，推动视频生成模型从研究走向商业应用。



闭源模型：Sora (OpenAI), Seedance (字节), Kling (快手), 海螺 (MiniMax), Vidu(生数科技)

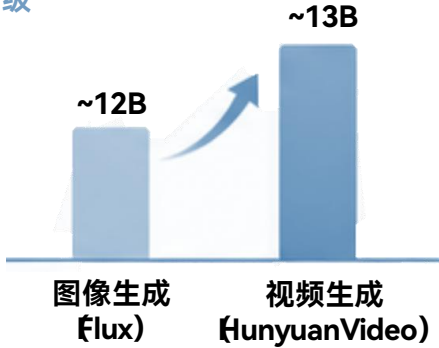


开源模型：HunyuanVideo (腾讯), Wan (阿里), CogVideoX (智谱)

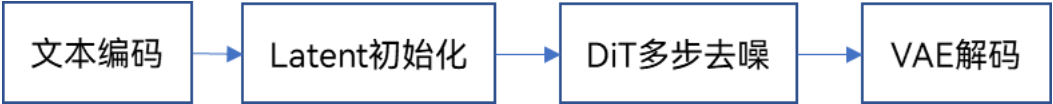
模型规模

模型	参数量	典型分辨率	典型时长
HunyuanVideo	13B	720P	5s
Wan2.2	14B	720P	5s
CogVideoX	5B	480P	6s

- 图像生成模型与视频生成模型参数相当
- 视频生成推理计算量高出一个数量级



视频生成推理流程



推理瓶颈

序列长度爆炸：
接近119K

720P 5秒视频，
Token数可达119K

推理步数叠加：
20~50 步

DiT架构需20~50步去噪，
每步完整执行一次
Transformer的前向传播

视频生成时间长：
~3500秒

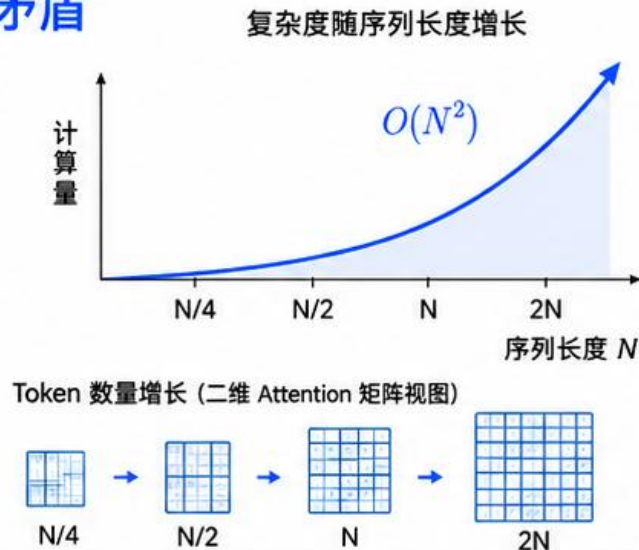
A3生成一段5s 720P视频耗时
达3500秒 (HunyuanVideo
未优化的开箱性能)

- 高昂的推理成本制约了视频生成模型的规模化部署
- 亟需在保持生成质量的前提下大幅提升推理效率

攻克 Attention 计算瓶颈：稀疏与量化的协同

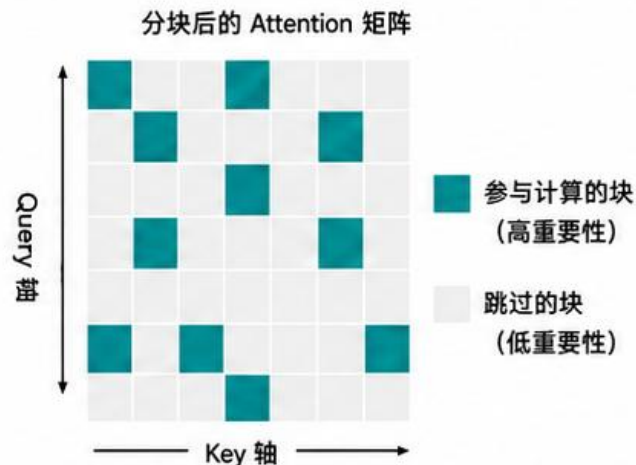
Attention 计算量是核心矛盾

- 视频生成模型中，Attention 的计算开销占主导
- $O(N^2)$ 复杂度随序列长度急剧膨胀，是整体延迟的主要来源
- 优化目标明确：减少 Attention 的实际计算量



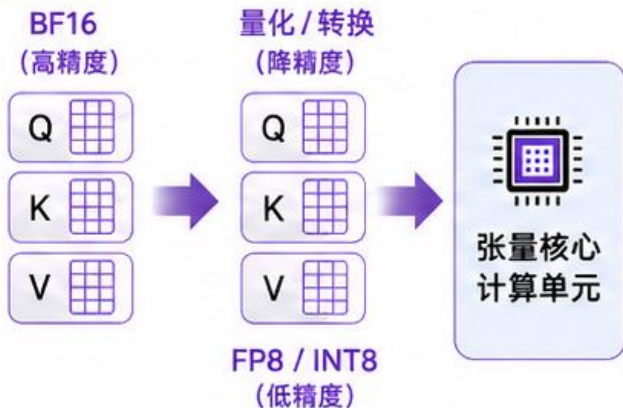
块稀疏注意力——减少参与计算的块

- 视频 Attention 矩阵天然稀疏，大量块内 & 点贡献极小
- 将 Attention 矩阵分块，识别并跳过低重要性块，直接减少 FLOPs
- 复杂度从 $O(N^2)$ 降至接近 $O(N)$ ，序列越长收益越大



量化——让每次计算更快

- 将 Attention 中 Q、K、V 的矩阵运算从 BF16 降至 FP8/INT8
- 低精度数据格式在 GPU/NPU 张量核心上吞吐量更高，相同数据量下完成计算更快
- Attention 的每个 tiling 块计算加快，整体计算耗时显著下降



两者协同

- 块稀疏：减少要算多少块
- 量化：让每块算得更快
- 共同作用于 Attention 计算，效果可叠加



本工作在视频生成场景下联合应用两种手段，从“算更少”和“算更快”两个维度压缩 Attention 计算量

目录

Part 1 背景

Part 2 块稀疏注意力

Part 3 低比特量化FA

块稀疏注意力效果

视频规格	方法	稀疏度	DiT 时间(s)	DiT 加速比	整网推理 时间(s)	整网推理 加速比
720*1280* 5s	baseline	/	3463	/	3587	/
	Top-K (范式1)	76% 80%步	2030	1.71x	2160	1.66x
	SVG (范式2)	80% 74%步	2100	1.65x	2260	1.59x

- Top-K方法：使能HunyuanVideo模型中最后40个Step，平均稀疏度为76%，DiT部分可取得1.71倍加速，整网加速比为1.66倍
- SVG方法：使能HunyuanVideo模型中最后37个Step，平均稀疏度为80%，DiT部分可取得1.65倍加速，整网加速比为1.59倍

稀疏视频效果

baseline



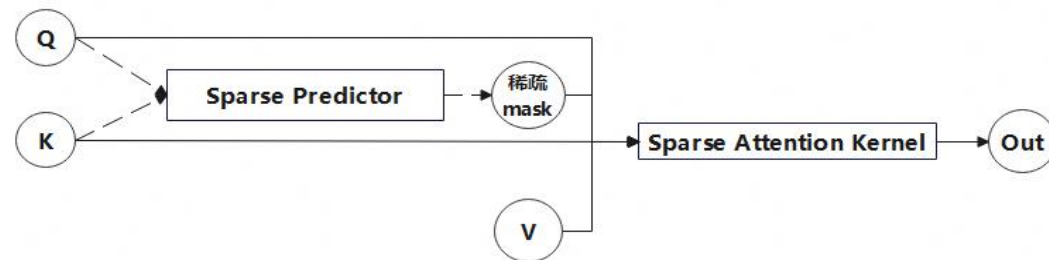
稀疏



块稀疏注意力方法总览

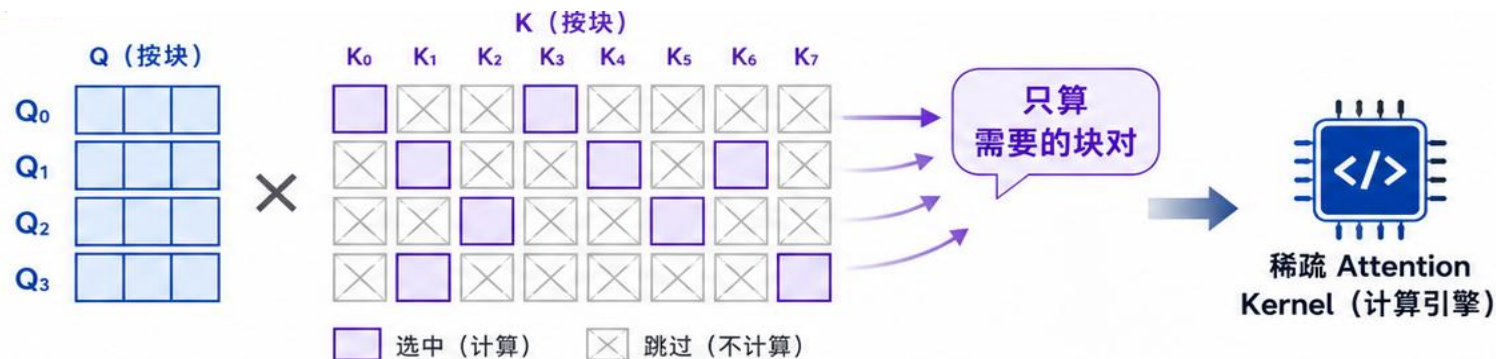
基于块稀疏注意力方案概览

- 分为两阶段操作：Predictor + Kernel
 - Predictor**: 基于注意力分布预测块稀疏模式，筛选高贡献分块并生成稀疏掩码剪枝冗余计算
 - Kernel**: 依据预测掩码执行稀疏化注意力运算



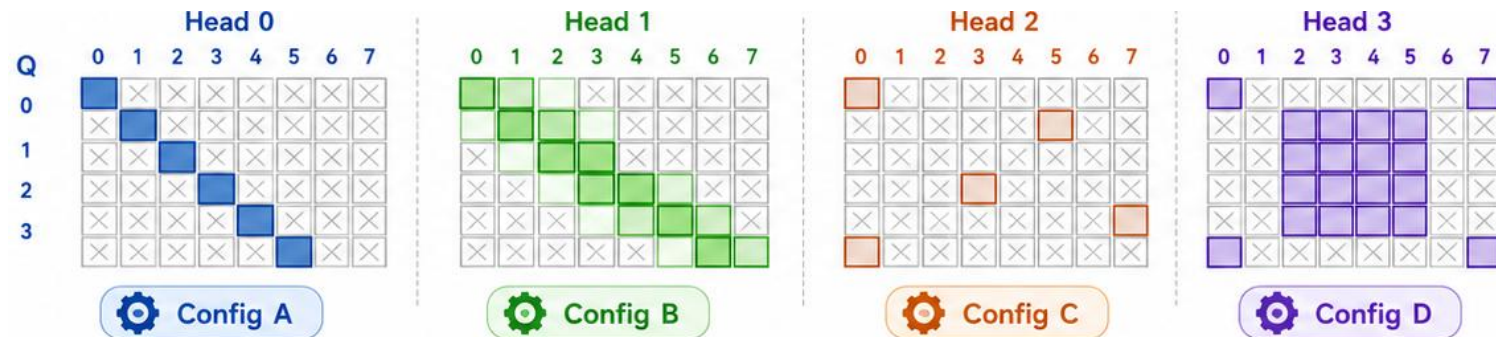
块稀疏注意力Kernel

分块粒度稀疏：支持Q-K块级计算跳过



- 根据传入的块稀疏掩码，仅执行被选中Q与K块间的计算
- 跳过未被选中的块对的MAC计算与对应访存，减少无效开销

逐头异构稀疏



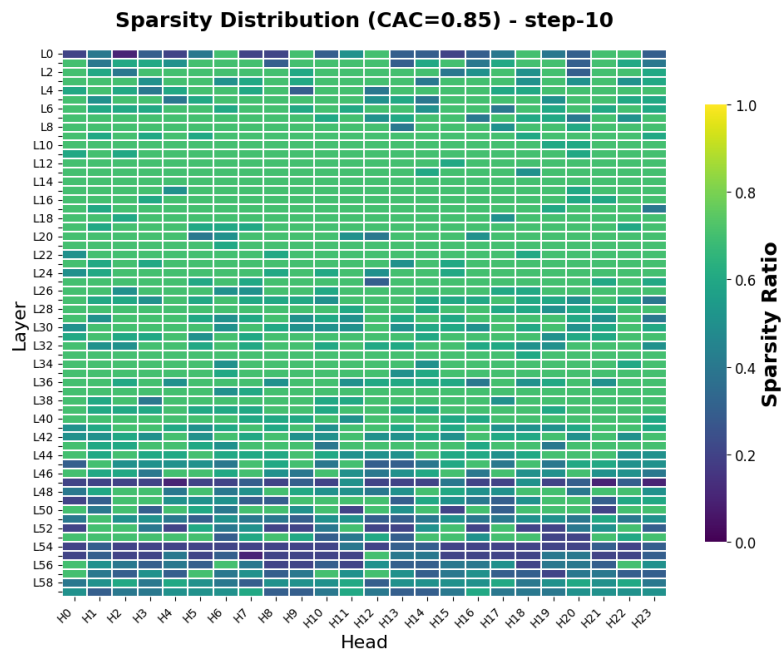
- 需支持不同注意力头采用独立的稀疏配置
- 允许各头按自身模式选择不同的块索引

基于上述特点，本方法使用CANN/ops-transformer提供的昇腾亲和的分块稀疏算子 [BlockSparseAttention](#)

块稀疏注意力方法总览

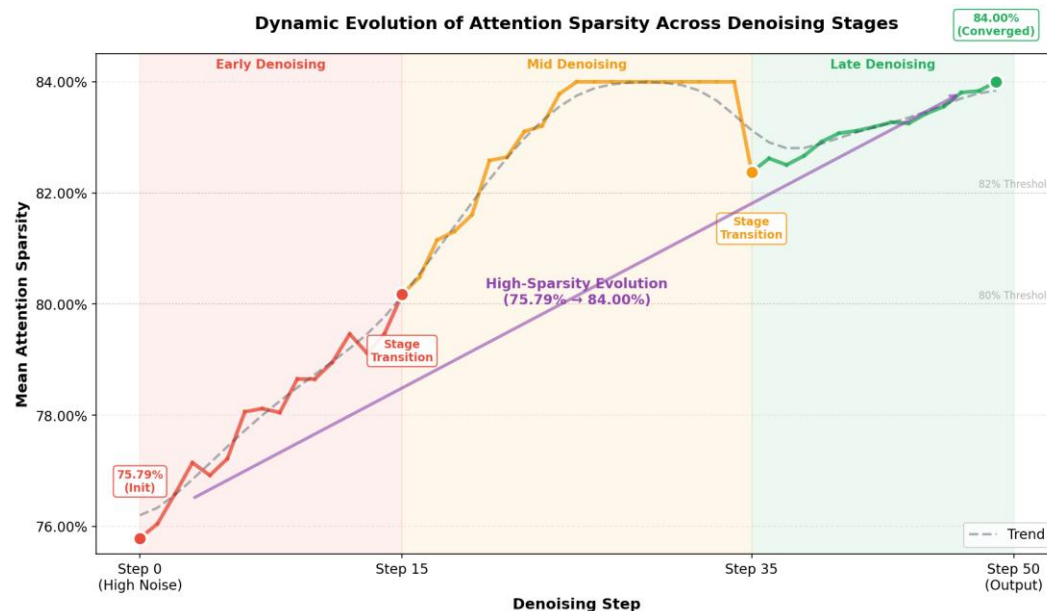
多模视频生成模型稀疏特征

跨层跨头稀疏程度差异



同一层内不同头，不同层之间的稀疏程度差异显著

稀疏度随时间步变化



同一头在不同时间步上的稀疏度也变化明显

Q1: 为什么要做头粒度差异化稀疏?

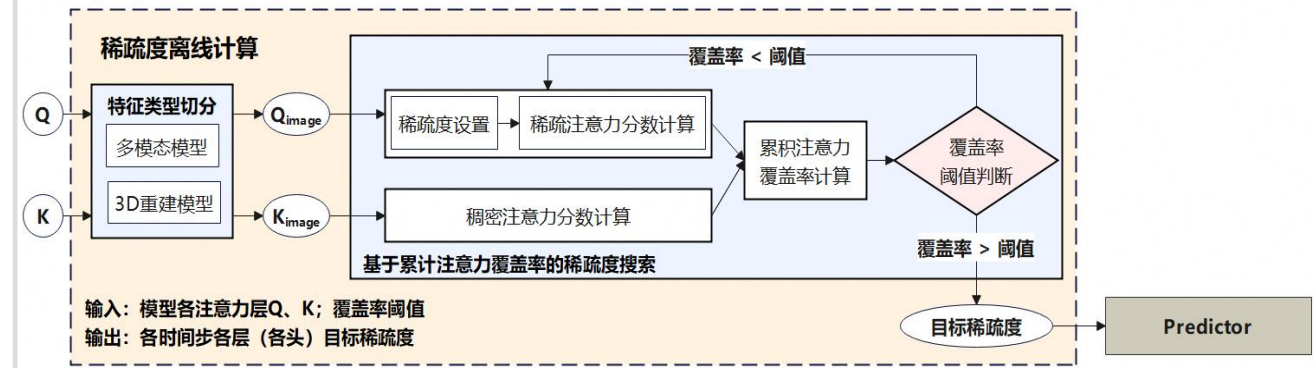
- 不同注意力头的稀疏潜力不同，统一稀疏度会限制整体收益，因此需要不同头采用不同稀疏度。

Q2: 为什么可以提前判断头的稀疏特征?

- 注意力头的稀疏度与提示词无关;
- 提示词主要影响重要元素位置。

块稀疏注意力Predictor方法一：TopK Predictor

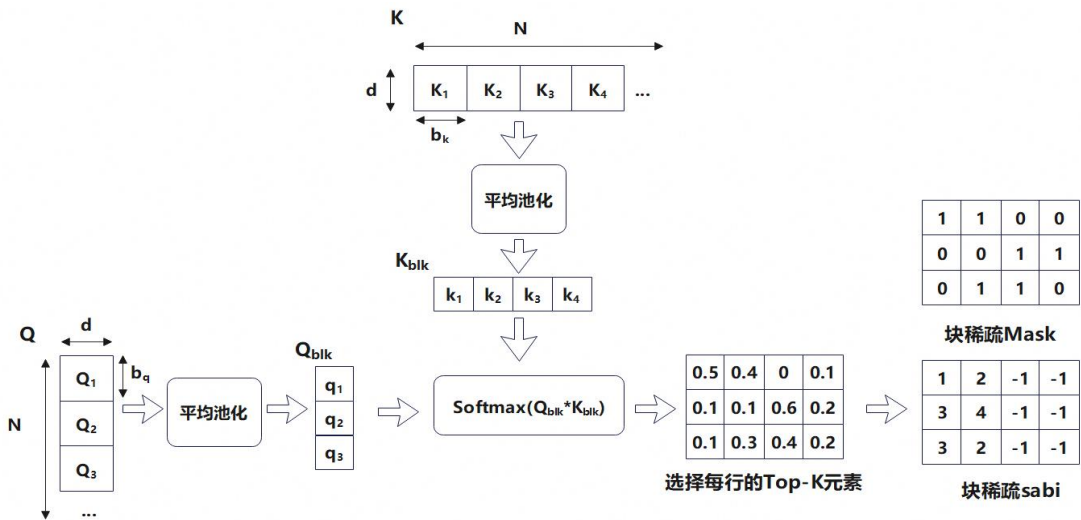
稀疏度离线计算 (Offline Profiling)



累积注意力覆盖率 (CAC) :
$$CAC(K) = \frac{\sum_{(i,j) \in Top-K} Softmax(QK^T/\sqrt{d_k})_{ij}}{\sum_{i,j} Softmax(QK^T/\sqrt{d_k})_{ij}}$$

- 稀疏度离线计算：基于累积注意力覆盖率的稀疏度搜索
 - ❑ 生成 Top-K 掩码：对每个 Query token，仅保留注意力得分最高的 Top-K 个 Key，形成稀疏注意力掩码。
 - ❑ 注意力覆盖率计算：在不同稀疏度配置下，计算累积注意力覆盖率 CAC_{eff}
 - ❑ 搜索最小可用稀疏度：遍历不同稀疏度，搜索到满足累积注意力覆盖率大于阈值的最小稀疏度，作为该头的稀疏度结果

稀疏Predictor方案1 (Top-K)：分块打分范式

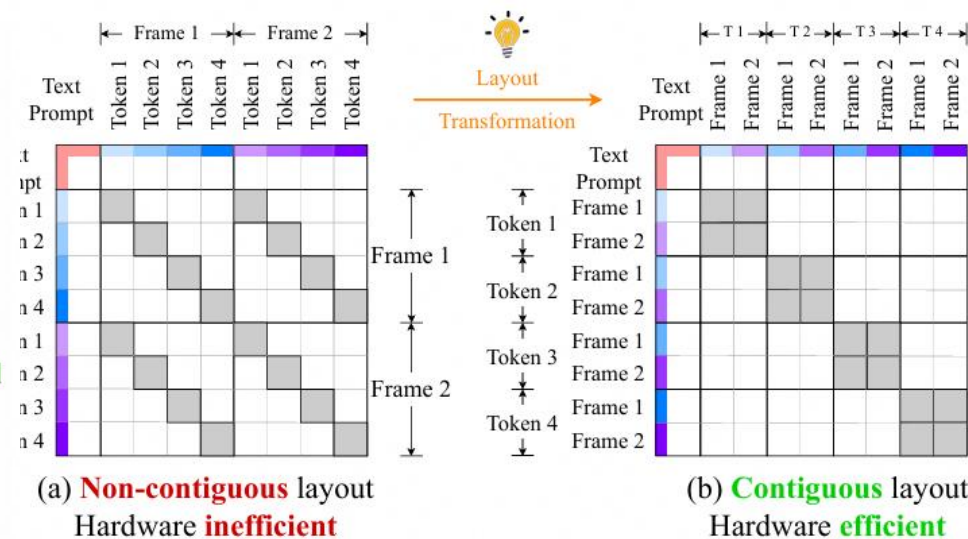
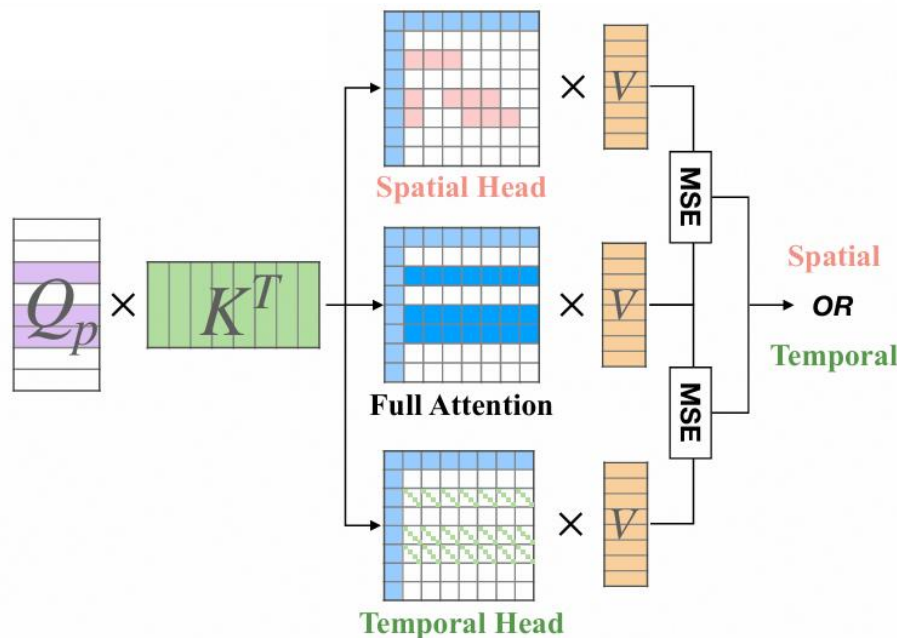
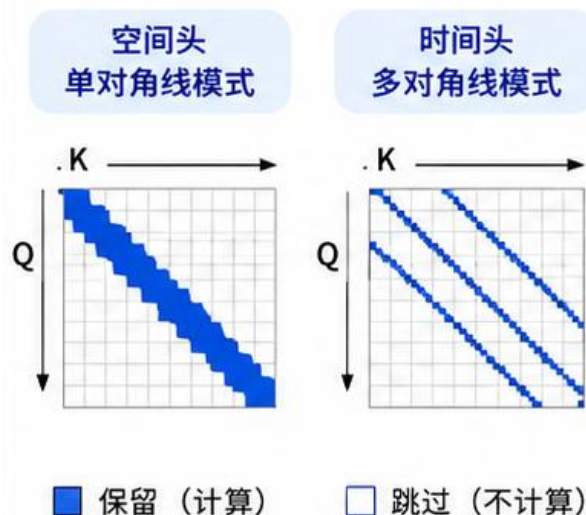


- 分块打分范式——TopK
 - ❑ Q \ K序列划分为多个连续分块，平均池化计算块级注意力得分
 - ❑ 基于块级注意力得分与各头稀疏度，筛选重要分块，生成稀疏掩码

块稀疏注意力Predictor方法二：SVG Predictor

稀疏Predictor方案2 (VG)：固定稀疏模式范式

稀疏模式



Token重排([F,H,W]->[H,W,F])

● 单对角线模式

- 关注同一序列块/当前帧内的邻近 token
- 对角线较粗，每个 Query 关注更多局部token

● 多对角线模式

- 关注当前 token 在其他帧上的信息，捕捉时序依赖
- 每条对角线较细，每个 Query 只关注若干关键 token

注意力头稀疏模式在线判别

● 固定稀疏模式范式——SVG

- 推理时在线判别稀疏模式，使用预定义的稀疏掩码
- 引入Token重排操作，将符合多对角线模式的token重排为单对角线，使稀疏块访问更加连续，从而支持硬件友好的规则化内存访问。

块稀疏注意力精度

当前稀疏方案精度									
视频规格	方法	块大小	稀疏度	Subject consistency	Motion Smoothness	Dynamic Degree	Aesthetic Quality	Imaging Quality	Overall consistency
720*1280*129帧	Top-K (范式1)	128(Q)*512(K)	76% 80%步	↓ 0.23	↓ 0.11	↓ 1.66	↓ 0.3	↓ 0.089	↑ 0.35
	SVG (范式2)	128(Q)*512(K)	80% 74%步	↓ 0.05	↓ 0.08	↑ 1.67	↓ 0.09	↓ 0.24	↑ 0.09

- 两种稀疏Attention方法精度符合标准

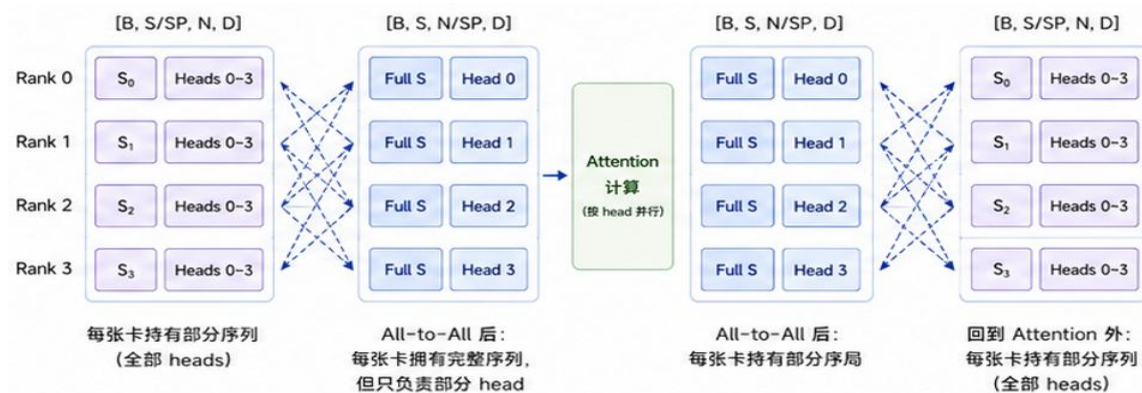
统一稀疏精度 vs 异构稀疏精度							
方法	稀疏度	Subject consistency	Motion Smoothness	Dynamic Degree	Aesthetic Quality	Imaging Quality	Overall consistency
所有头使用统一稀疏度	76%	↓ 0.16	↓ 0.07	↓ 0.83	↓ 0.28	↓ 0.24	↑ 0.42
所有头使用不同稀疏度	均值76% (57%头84%)	↓ 0.23	↓ 0.11	↓ 1.66	↓ 0.3	↓ 0.089	↑ 0.35
所有头使用统一稀疏度	84%	↓ 0.44	↓ 0.2	↓ 0.83	↓ 0.73	↓ 1.56	↑ 0.5

- 所有头使用统一的稀疏度并不能很好适应不同注意力头之间的差异。
- 不同头使用不同的稀疏度能够提高整体稀疏度，保证更高稀疏度下的精度。

稀疏与序列并行结合

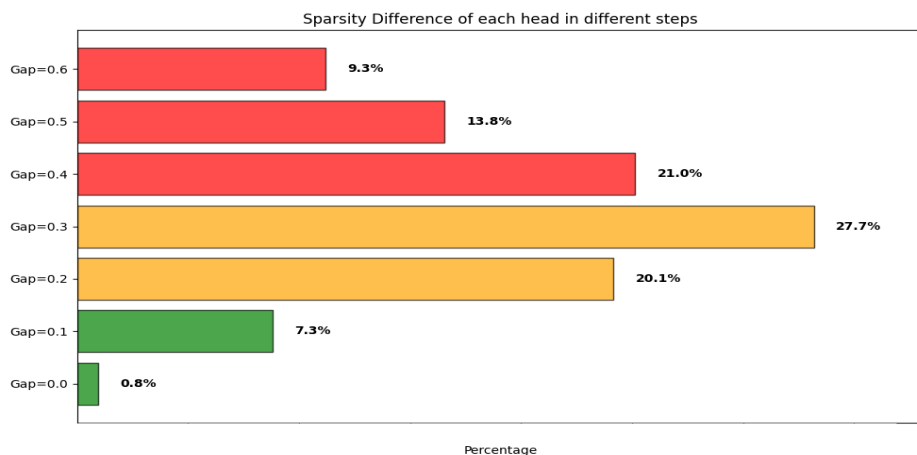
面向长序列注意力的序列并行方法 Ulysses

- 长序列场景下，单卡难以承载完整序列注意力计算，Ulysses通过序列并行将输入序列切分到多张设备上，基于卡间通信实现多卡并行注意力计算
- Ulysses序列并行核心：**序列维度在设备间切分，每个头在单设备内独立计算



块稀疏注意力叠加 Ulysses并行的执行拖尾问题

- 真实推理中，同一层头间稀疏度有差距。不同时间步、不同层、不同头的稀疏度不同且持续变化。



HunyuanVideo网络同层内头间稀疏度差距

<https://gitcode.com/cann>

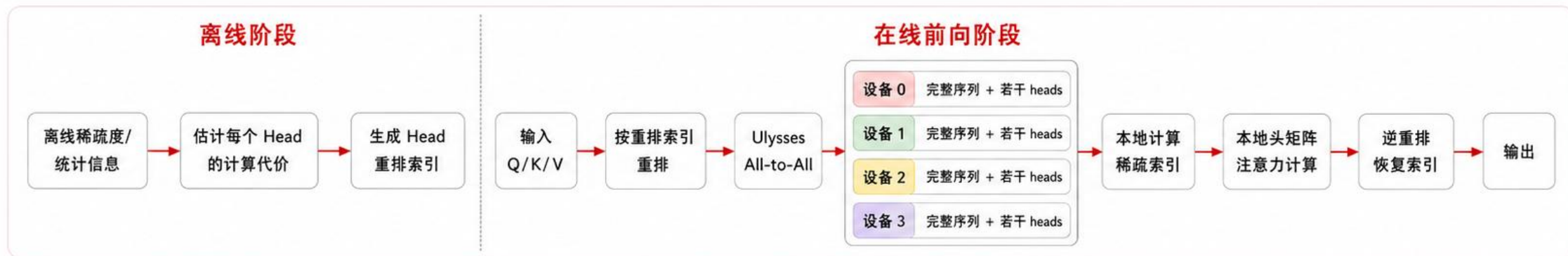
- Ulysses 序列并行中，不同注意力头被分配到不同设备独立计算。
- 各头稀疏度存在差别，并行注意力计算时延由计算负载最重的设备决定。即便整体平均稀疏度较高，推理过程仍可能被“慢卡”拖住



CANN

基于头重排的 Ulysses 负载均衡

按头重排，均衡每设备计算量



基于离线稀疏度估计各注意力头计算
代价，并生成重排索引

前向计算时按该索引重新分配注意力头，实现多设备负载均衡。

HunyuanVideo 上块稀疏叠加序列并行的性能结果

- 多卡场景下，未做负载均衡时整体性能受最慢设备限制，端到端推理时间为 **335 s**，实现 **1.39×** 加速；引入注意力头索引重排后，推理时间进一步降至 **321 s**，额外提升约 **6%**，整体加速达 **1.45×**。

方法	单卡		无负载均衡 (8卡)		负载均衡 (8卡)	
	整网时间 (s)	加速比	整网时间 (s)	加速比	整网时间 (s)	加速比
baseline	3587	/	464	/	464	/
所有头使用 不同稀疏度	2160	1.66x	335	1.39x	321	1.45x

稀疏视频效果

baseline



稀疏



<https://gitcode.com/cann>



目录

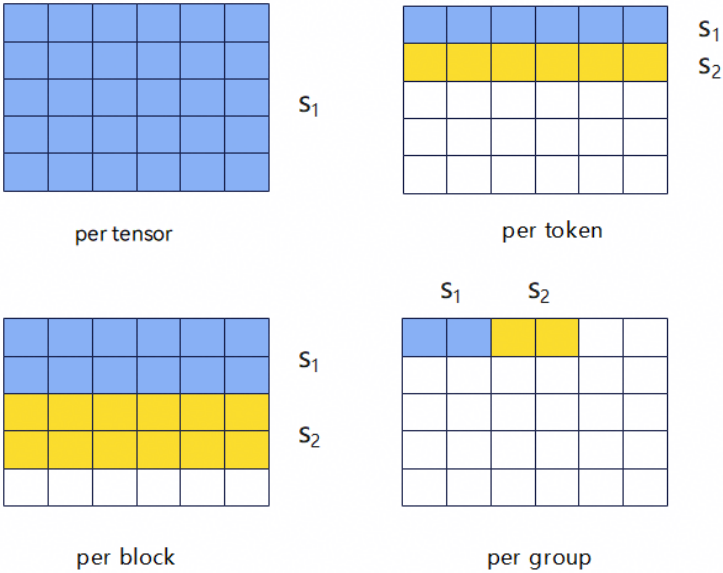
Part 1 背景

Part 2 块稀疏注意力

Part 3 低比特量化FA

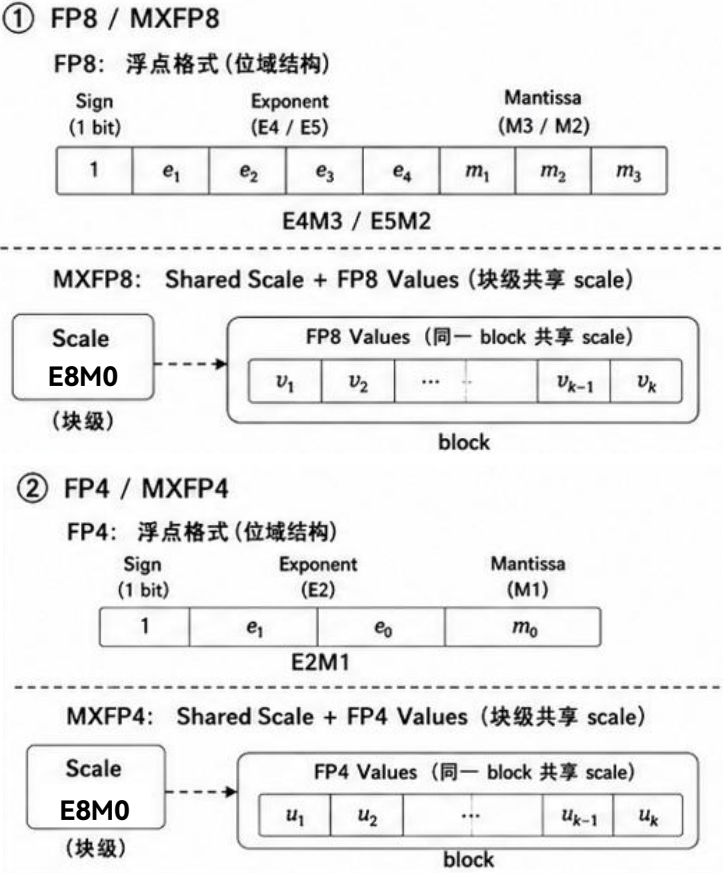
量化关键技术：粒度、量化数据格式与误差抑制

量化粒度



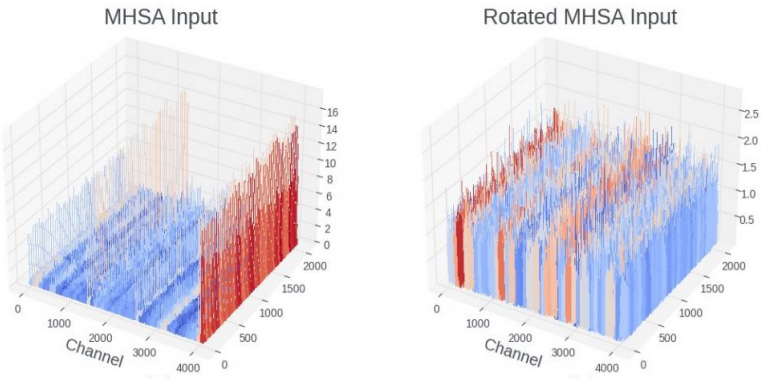
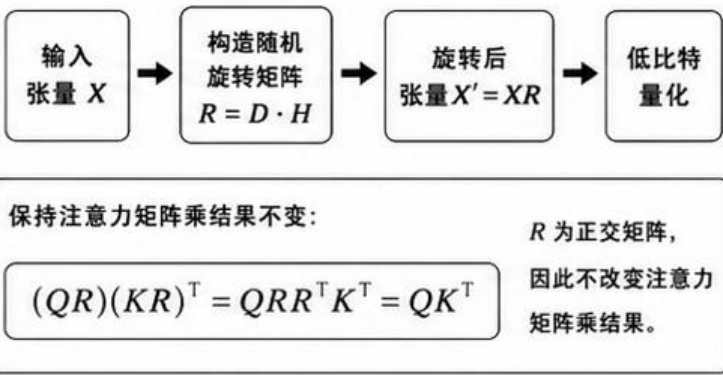
粒度越细，量化误差越小，单scale存储和计算开销越大

MXFP低精度数据格式



MXFP 细粒度分组提升低比特表达能力，E8M0 scale 降低反量化开销

误差抑制：RHT



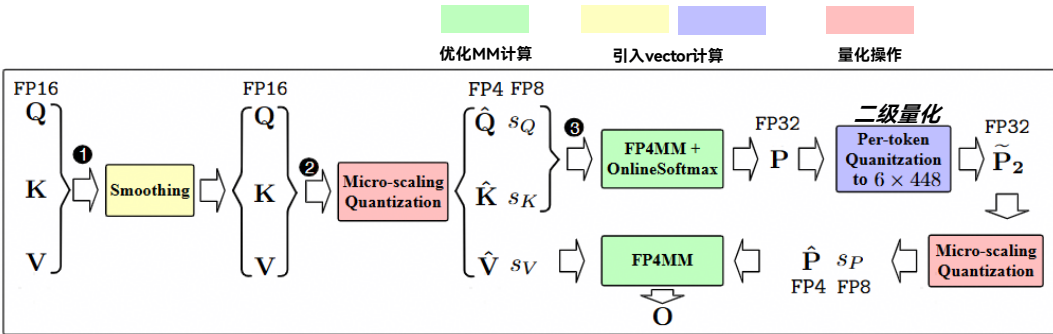
RHT 通过随机正交旋转打散离群值，使得数据分布更加平滑以降低量化误差

SageAttention 3 NPU 性能分析

SageAttn3 加速效果



SageAttn3 量化方案



SageAttn3是社区关注度比较高的 FA 加速方案。其将主体计算推进到全 4bit，在 RTX5090平台拿到了明显的加速收益，在多模领域精度损失可控。

性能优化方法:

- FP4 Tensor Core 加速 QK / PV Matmul 计算
- P量化和online softmax 复用 rowmax 节省计算开销
- K/P布局重排与 pipeline 优化

精度优化方法:

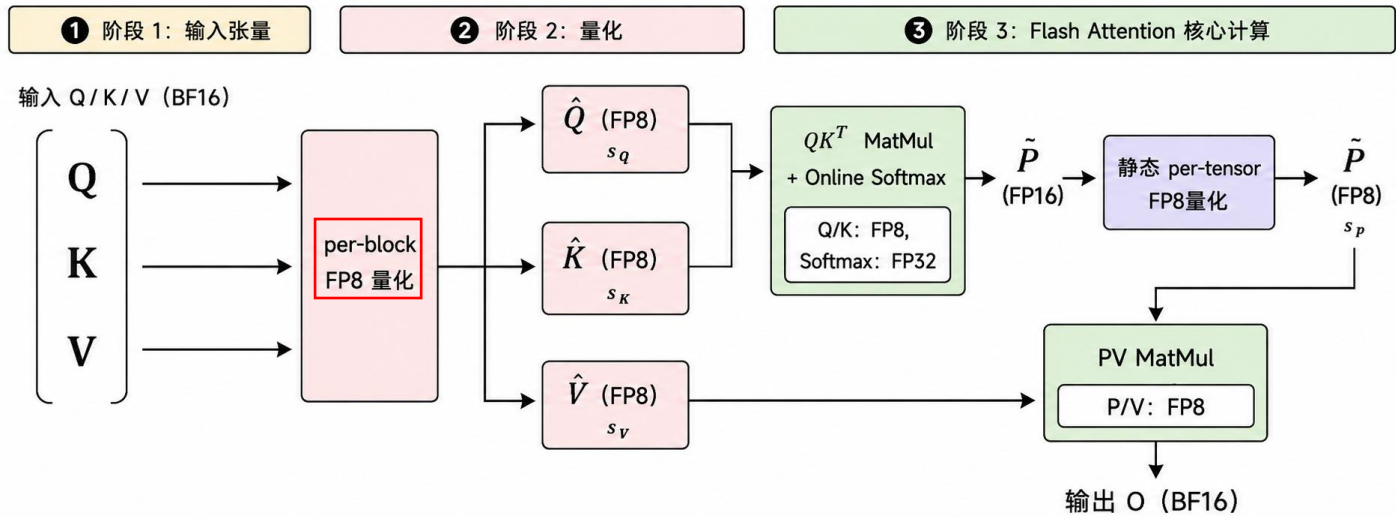
- 减均值smoothing：QKV减去各自channel维度的均值抑制outlier
- 二级量化：将P从[0,1] 放大至 [0,6x448] 数值范围

NPU亲和度不高:

- NPU下 bf16 FA 是Cube计算Bound，最优 fp8 FA 达到 Cube/Vector 计算双Bound
- SageAttn3方案 FP4 Matmul 可降低 Cube 计算量，但 FA 的整体耗时还受 online_softmax、update、量化等 Vector 侧开销影响。此外 smoothing 和 二级量化使 Vector 计算不降反增
- SageAttn3方案平移迁至昇腾NPU，相比 FP8 FA 未必有明显加速收益

低比特量化 FA 方案穿刺

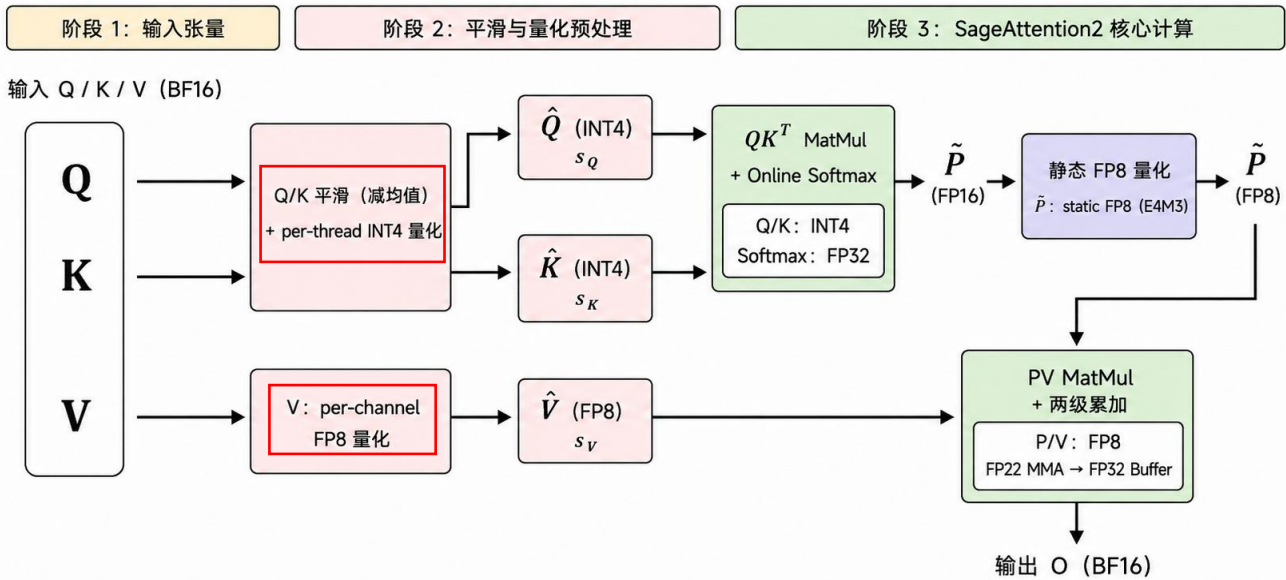
FP8 Flash Attention 量化方案



FP8 量化 FA 方案：

- FP8_e4m3 有256个数表示能力足够，Q/K/V 采用block粒度量化精度损失较小
- Per-Block 量化方式契合 FA tiling 分块策略，一次UB搬运可完成反量化，做到反量化代价最小

SageAttention2 量化方案



4/8 混精量化 FA 方案：

- Sageattn2 采用 4/8 混精设计，贴 GPU 特性的 per_thread int4 量化做到了性能显著提升且无明显精度损失
- 对应的我们改进了一版 NPU 亲和的 4/8 混精量化 FA 方案，4bit 减 Cube 计算、结合 vector 计算优化，拿到明显的性能加速收益

低比特量化 FA 精度和性能结果

在 hunyuanvideo 上使能量化 FA 做精度和性能评测

精度实测

量化格式	量化方案	subject consistency	motion smoothness	dynamic degree	aesthetic quality	imaging quality	overall consistency
F8 FA	Q/K/P/V Block 粒度量化	↑0.1	↑0.02	↓1.67	↓0.02	↓0.18	↓0.04
4/8混精	量化直转	↓1.0	↓0.17	↑6.66	↓1.58	↓4.03	↓0.41
	量化算法优化	↓0.08	↑0.05	↓0.84	↓0.26	↓0.89	↓0.47
	量化算法优化 + vector 计算优化	↑0.04	↑0.05	↓0.84	↓0.12	↓0.97	↓0.31

性能实测

FA实现	Dit 耗时 / s	整网耗时 / s	整网加速比
BF16 FA	2548.49	2580.87	-
FP8 FA	1606.29	1638.55	1.58



tips: 4bit量化目前无法做到量化前后肉眼一致，主要看整体语义相似性和视频质量

收益评估：FP8 FA量化方案保持较高的精度，且有显著的加速效果；4/8混精FA量化方案精度达标，优化vector计算后评估可拿到加速收益。

欢迎大家扫码加入我们的社区



cann recipes infer



微信交流群



飞书交流群



cann amct

技术交流 & 在线答疑 & 行业动态 & 资源分享

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯